



Conceptos clave — Proyecto Jago (para programadores)

Resumen técnico de lo **indispensable** para entender y operar el proyecto en **local, web y celular**. Pensado para explicar rápido.

1) Conceptos base (los que hay que dominar)

Concepto	En una frase	En este proyecto
Frontend	Código que corre en el navegador (HTML/CSS/JS)	Toda la página de Jago
Sitio estático	Solo archivos (sin servidor propio); el navegador hace todo	Jago es estático → por eso se publica gratis
Backend / servidor	Código que corre en un servidor (recibe peticiones, usa BD)	Jago no tiene backend propio (usa servicios)
Cliente ↔ Servidor	El cliente (navegador) pide; el servidor responde	Jago (cliente) llama a Gemini y Supabase (servidores)
API	"Enchufe" para que dos programas se comuniquen	Web Speech API (voz), Gemini API (IA), Supabase API (BD)
API Key	Contraseña que identifica tu cuenta al usar una API	La key de Gemini; nunca se publica
Variables de entorno (.env)	Dónde se guardan secretos fuera del código	Claves del servidor (no se suben a Git)
Dominio / URL	La dirección web (misitio.com)	hays05.github.io/feria-jaguar
DNS	"Directorio" que traduce dominio → servidor	Solo importa si compras dominio propio
Hosting	Dónde "vive" la página para que esté en internet	GitHub Pages
Git / GitHub	Control de versiones + repositorio en la nube	Guarda el código y dispara la publicación
HTTPS	Conexión segura (candado)	Obligatorio para micrófono y voz
Base de datos	Dónde se guardan los datos	Local (localStorage) y nube (Supabase)
RLS	Reglas de quién puede leer/escribir en la BD	En Supabase: solo permite insertar
CORS	Permiso para llamar a otro dominio desde el navegador	Gemini y Supabase ya lo permiten

2) Para que corra en LOCAL (tu computadora)

Indispensable: - **Un servidor local** (no abrir el `.html` con doble clic). Motivo: el navegador bloquea cargar archivos y el micrófono si no hay servidor. `python -m http.server 8000` → `http://localhost:8000` (o la extensión **Live Server** de VS Code). - **Navegador Chrome/Edge** (mejor soporte de voz). - `localhost` **cuenta como seguro** → el micrófono funciona aunque no sea HTTPS.

Resumen: **servidor local + navegador**. Nada más para probar.

3) Para que corra en WEB (publicado en internet)

Indispensable: un **hosting**. Dos opciones según el tipo de proyecto:

	GitHub Pages (lo que usa Jago)	Vercel / Netlify
Para qué	Sitios estáticos (HTML/CSS/JS)	Apps con servidor/backend (Next.js, APIs)
Costo	Gratis	Gratis (plan hobby)
Cómo publica	Conecta el repo de GitHub → git push actualiza	Conecta el repo → git push actualiza
HTTPS	Automático	Automático
Variables de entorno	No (es estático)	Sí (para secretos del servidor)

Flujo de publicación (Jago):

```
git push → GitHub Pages reconstruye → https://usuario.github.io/proyecto
```

Regla práctica: **estático** → **GitHub Pages**. **Con backend/secretos del servidor** → **Vercel**. (La quiniela del colegio usa **Vercel** porque sí tiene backend; Jago no.)

4) Para que corra en CELULAR

Indispensable (no hace falta una "app", funciona en el navegador del teléfono): - **HTTPS obligatorio**: el micrófono y la voz **solo funcionan en https://** en el celular. Por eso se publica (GitHub Pages da HTTPS). - **Diseño responsive**: la etiqueta `<meta name="viewport" ...>` + CSS flexible para que se adapte a pantallas chicas. - **Acceso fácil**: un **código QR** que apunta a la URL → el visitante escanea y abre Jago al instante. - **Permisos**: el celular pide permiso de **micrófono** la primera vez.

Resumen: **HTTPS + responsive + QR**. Mismo sitio web, sin instalar nada.

5) Base de datos (lo esencial)

Qué es: el lugar donde se **guardan los datos** para no perderlos. Dos tipos según dónde viven:

	Local (en el dispositivo)	En la nube (servidor)
Ejemplo aquí	localStorage del navegador	Supabase (PostgreSQL)
Dónde guarda	Solo en ESA compu/celular	En internet, central
¿Se juntan todos?	No (cada aparato lo suyo)	Sí, todos en un lugar
Cuándo usar	Kiosco / 1 sola compu / offline	Muchos dispositivos, ver desde cualquier lado

Términos que hay que conocer: - **Tabla**: como una hoja de Excel (ej. **visitantes**, **mensajes**). - **Fila (registro)**: una persona/un dato. **Columna (campo)**: un dato de esa fila (nombre, grado...). - **Consulta (query)**: pedirle algo a la BD (insertar, leer, borrar). En BD de nube se usa **SQL**. - **SQL / PostgreSQL**: el lenguaje y el motor de base de datos que usa Supabase. - **CRUD**: las 4

operaciones — **Crear**, **Leer** (Read), **Actualizar** (Update), **Borrar** (Delete). - **API REST de la BD**: Supabase expone la base por HTTP → el navegador guarda con un **POST** (no hace falta servidor propio).

Seguridad de la BD (clave): - **RLS (Row Level Security)**: reglas por tabla. En Jago: solo se permite **INSERT** (guardar); nadie puede leer/borrar desde la página. - **Llaves de Supabase:** - **publishable / anon key** → **pública** (va en el navegador); segura gracias a RLS. - **service_role key** → **secreta** (acceso total); SOLO en servidor, **jamás** en el navegador.

En Jago: cada visitante se guarda con un **POST** a Supabase usando la **publishable key** + RLS solo-insertar. El profesor lee/exporta desde el panel de Supabase.

✓ **Lo mínimo, en una línea por escenario**

- **Local:** servidor local (**localhost**) + navegador.
- **Web:** repositorio en GitHub + hosting (GitHub Pages estático / Vercel si hay backend) + HTTPS.
- **Celular:** la misma URL en HTTPS + diseño responsive + QR para abrirla.

🔌 **Servicios externos que usa Jago (vía API)**

Servicio	Tipo	Para qué	Clave
Web Speech API	Navegador	Oír y hablar	No necesita
Google Gemini	API en la nube	IA que responde	API Key (privada)
Supabase	BD en la nube	Guardar visitantes	Publishable key (pública) + RLS
GitHub Pages	Hosting	Publicar	No (usa tu repo)